Front-End Web Technology
(2504CS305)

Unit 2: DOM and ES6

Complete Exam-Ready Study Notes

Covers: Document Object Model, Event Handling, DHTML, ES6 Variables, Arrow Functions, Array Methods, Destructuring, Spread/Rest, Classes and Modules

Table of Contents

1. Document Object Model (DOM)
2. Document Object Properties and Methods
3. Accessing HTML Elements
4. Forms Using DOM
5. DHTML
6. HTML Element Properties
7. Event Handling with JavaScript
8. ES6 Overview
9. Variables: var, let and const
10. Arrow Functions
11. ES6 Array Methods
12. Destructuring
13. Spread and Rest Operators
14. Classes in JavaScript
15. Modules
16. Final Revision
17. Exam Practice Questions and Answer Key

1. Document Object Model (DOM)

Simple definition: The Document Object Model (DOM) is a programming interface that lets JavaScript read and change the content, structure, and style of an HTML document.

Detailed explanation: When a browser loads an HTML page, it creates a tree-like structure called the DOM. Every HTML tag, attribute, and piece of text becomes an object (node) in this tree. JavaScript can use the DOM to find elements, change their text or attributes, add/remove elements, and respond to user actions such as clicks.

Key points:

- DOM is platform- and language-neutral: it can be used by JavaScript and other languages.
- The **window** object is the global object in client-side JavaScript and the main entry point to browser features and the document.
- document is actually a property of window, so document.write("Hello World") is the same as window.document.write("Hello World").
- The window object represents the browser window or frame displaying the document.
- All client-side objects are connected to the window object.

Diagram explanation (DOM tree): The browser converts HTML into a tree. **window** is at the top; below it is **document**; document contains **html**; html contains **head** and **body**; body contains elements such as headings, paragraphs, images, and forms.

```
window
■■■ document
■■■ html
■■■ head
■■■ body
■■■ h1
■■■ p
■■■ form
```

Example: If a page has Hello , JavaScript can change its text using document.getElementById("msg").innerHTML = "Welcome";

Important exam points: DOM = interface to access/update document content, structure and style; window is the global object; document is part of window.

2. Document Object Properties and Methods

The document object provides properties to get information about the page and methods to access or modify elements.

Document object properties

Property	Description
anchors	Returns a collection of all anchors in the document (deprecated).
applets	Returns a collection of all applets in the document (deprecated).
body	Returns the body element of the document.
cookie	Returns all name/value pairs of cookies in the document.
domain	Returns the domain name of the server that loaded the document.
forms	Returns a collection of all forms in the document.
images	Returns a collection of all images in the document.

links	Returns a collection of all links in the document.
referrer	Returns the URL of the document that loaded the current document.
title	Sets or returns the title of the document.
URL	Returns the full URL of the document.

Note: In the original material, anchors, applets and links were marked as deprecated/unsupported in many browser versions.

Document object methods

Method	Description
write()	Writes HTML expressions or JavaScript code to the document.
writeln()	Same as write(), but adds a newline character after each statement.
open()	Opens an output stream to collect output from write() or writeln().
close()	Closes the output stream opened by document.open().
getElementById()	Accesses one element with the specified id.
getElementsByName()	Accesses all elements with the specified name.
getElementsByTagName()	Accesses all elements with the specified tag name.
setTimeout()	Calls a function once after a set time period.
clearTimeout()	Cancels a timeout set by setTimeout().

3. Accessing HTML Elements

JavaScript needs a reference to an HTML element before it can change it. The three main methods are getElementById(), getElementsByName() and getElementsByTagName().

getElementById()

Returns the single element whose id matches the given value. The innerHTML property is commonly used to read or replace the content of an element.

```
HTML:
<input type="text" id="demo">

JavaScript:
document.getElementById("demo").innerHTML = "Hello World!";
```

Working: 1) Browser finds the element with id="demo". 2) JavaScript stores a reference to it. 3) innerHTML replaces the content with "Hello World!".

getElementsByName()

Returns a collection (array-like list) of all elements having the given name attribute.

```
HTML:
<input name="animal" type="checkbox" value="Cats">
<input name="animal" type="checkbox" value="Dogs">

JavaScript:
const collection = document.getElementsByName("animal");
for (let i = 0; i < collection.length; i++) {
if (collection[i].type == "checkbox") {
collection[i].checked = true;
}
}
```

Working: The loop checks every element returned for name="animal" and marks each checkbox as checked.

getElementsByTagName()

Returns a collection of all elements with the given tag name, such as "input", "p", or "img".

```
HTML:
<input type="text" name="uname">
<input type="text" name="pword">

JavaScript:
function myFunction() {
let a = document.getElementsByTagName("input");
alert(a[0].value);
alert(a[1].value);
}
```

Comparison of access methods

Method	Searches by	Returns	Use when
getElementById()	id	One element	The element has a unique id.
getElementsByName()	name	Collection of elements	Several elements share a name, e.g. checkboxes.
getElementsByTagName()	tag name	Collection of elements	You want all elements of one type, e.g. all inputs.

Common mistake: getElementById returns one element; getElementsByName and getElementsByTagName return collections, so you must use an index such as [0] to access one item.

4. Forms Using DOM

Form elements can be accessed easily using the name or id of the form. This is useful for validation, such as checking a username and password.

```
HTML:
<form name="myForm">
<input type="text" name="uname">
<input type="text" name="pword">
<input type="button" onClick="f()">
</form>

JavaScript:
function f() {
var a = document.forms["myForm"];
var u = a.uname.value;
var p = a.pword.value;
if (u == "admin" && p == "123") {
alert("valid");
} else {
alert("Invalid");
}
}
```

Step-by-step working: 1) The button click calls f(). 2) document.forms["myForm"] gets the form. 3) a.uname.value reads the username; a.pword.value reads the password. 4) The if condition compares both values and shows the appropriate alert.

Exam point: form elements are accessed as formName.elementName.value.

5. DHTML – Combining HTML, CSS and JavaScript

Simple definition: DHTML (Dynamic HTML) is the combination of HTML, CSS, and JavaScript used to create interactive and dynamic web pages.

Explanation: HTML gives the structure, CSS gives the appearance, and JavaScript makes the page respond to user actions. DHTML is commonly associated with animation and dynamic effects. A major historical problem with DHTML was browser compatibility.

- Get a reference to an HTML/CSS element using getElementById(), getElementsByName(), or getElementsByTagName().
- Change its attributes or style using reference.attribute syntax.
- Example: change an image using a.src = "xyz.jpg";

```
HTML:
<img src="abc.jpg" id="myImg">

JavaScript:
var a = document.getElementById("myImg");
a.src = "xyz.jpg";
```

DHTML colour-animation example

```
<div id="myDiv">Red Alert !!!!!!</div>

<script>
var objDiv = document.getElementById("myDiv");
var colors = ["white","yellow","orange","red"];
var nextColor = 0;
setInterval(
"objDiv.style.backgroundColor = colors[nextColor++ % colors.length];",
500
);
</script>
```

Working: Every 500 milliseconds, setInterval runs the code. The expression nextColor++ % colors.length selects colours in rotation: white, yellow, orange, red, then repeats. The background colour of the div keeps changing, creating an animation effect.

6. HTML Element Properties

After getting an element reference, JavaScript can read or change its properties.

Property	Description
className	Sets or returns the class attribute of an element.
id	Sets or returns the id of an element.
innerHTML	Sets or returns the HTML contents (including text) of an element.
style	Sets or returns the style attribute of an element.
tabIndex	Sets or returns the tab order of an element.
title	Sets or returns the title attribute of an element.
value	Sets or returns the value attribute of an element.

Example: obj.style.backgroundColor changes CSS; obj.value reads or sets an input value; obj.innerHTML changes the HTML inside an element.

7. Event Handling with JavaScript

Simple definition: An event is an action that happens in the browser, such as a click, key press, or form submission. Event handling means writing JavaScript code that runs when such an action occurs.

Mouse events

Event : Attribute	When it occurs
click : onclick	User clicks an element.
dblclick : ondblclick	User double-clicks an element.
mousedown : onmousedown	User presses a mouse button over an element.
mousemove : onmousemove	User moves the mouse pointer over an element.
mouseover : onmouseover	User moves the mouse over an element.
mouseup : onmouseup	User releases a mouse button over an element.
mouseout : onmouseout	User moves the mouse pointer out of an element.

Keyboard events

Event : Attribute	When it occurs
keydown : onkeydown	User presses or holds down a key.
keypress : onkeypress	User presses or holds down a key.
keyup : onkeyup	User releases a keyboard key.

Frame/object events

Event : Attribute	When it occurs
abort : onabort	Image loading is stopped before completion (for object).
error : onerror	An image does not load properly.
load : onload	A document, frameset, or object has loaded.
resize : onresize	The document view is resized.
scroll : onscroll	The document view is scrolled.
unload : onunload	Document is removed from a window/frame (deprecated).

Form events

Event : Attribute	When it occurs
blur : onblur	A form element loses focus.
change : onchange	Content, selection, or checked state changes.
focus : onfocus	An element gets focus.
reset : onreset	A form is reset.
select : onselect	User selects some text.
submit : onsubmit	A form is submitted.

Example: <button onclick="alert('Clicked!')">Click Me</button> shows an alert whenever the button is clicked.

Common confusion: onblur occurs when an element loses focus, while onfocus occurs when it gains focus.

8. ES6 Overview

ES6 (ECMAScript 2015) is a major update to JavaScript that added cleaner syntax and powerful features. The important features covered in this unit are:

- var vs let vs const (variables and scoping)
- Objects and this

- Arrow functions
- Array functions/methods
- Destructuring
- Spread operator
- Classes
- Modules

Why ES6 matters: It makes JavaScript code shorter, safer, easier to organize, and more similar to modern object-oriented programming styles.

9. Variables: var, let and const

Simple definition: A variable is a named space in memory that stores a value. Variable names are called identifiers.

A variable must be declared before use. ES5 used var; ES6 introduced let and const.

let keyword

- Introduced in ES6 (2015).
- Variables declared with let have **block scope**.
- Must be declared before use.
- Cannot be redeclared in the same scope.
- A let variable inside { } cannot be accessed outside that block.

```
function order(x, y) {
if (x > y) { // block starts
let temp = x;
x = y;
y = temp;
}
console.log(temp === x); // ReferenceError: temp is not defined
return [x, y];
}
```

Explanation: temp exists only inside the if block. Outside the block, JavaScript cannot find it, so it gives a ReferenceError.

const keyword

- Introduced in ES6 (2015).
- A const variable cannot be redeclared or reassigned.
- It must be assigned a value when declared.
- It has block scope.
- const defines a constant **reference**, not necessarily an unchangeable value.

```
const x; // SyntaxError: missing = in const declaration
const y = 123;
y = 456; // TypeError: "y" is read-only

for (const x of ["a","b"]) {
console.log(x);
}
// Output:
```

```
// a
// b
```

Important: You cannot reassign a const array or object, but you CAN change its elements or properties.

```
const arr = [1, 2, 3];
arr.push(4); // Allowed: changes contents
// arr = [5, 6]; // Error: reassignment not allowed

const person = { name: "Amit" };
person.name = "Riya"; // Allowed: changes property
// person = {}; // Error: reassignment not allowed
```

var vs let vs const

Feature	var	let	const
Introduced	ES5	ES6	ES6
Scope	Function scope	Block scope	Block scope
Redeclaration in same scope	Allowed	Not allowed	Not allowed
Reassignment	Allowed	Allowed	Not allowed
Initial value required	No	No	Yes
Typical use	Old code	Value that can change	Value that should not be reassigned

Common mistake: Students often think const makes arrays/objects completely unchangeable. It only prevents reassignment of the variable; the contents can still change.

10. Arrow Functions

Simple definition: An arrow function is a shorter way to write a JavaScript function using the arrow symbol **=>**. It is also called a lambda function or fat-arrow function.

- Arrow functions are anonymous functions: they do not have a name bound to an identifier.
- They do not use the function keyword.
- They cannot be used as constructors.
- They have no prototype property and cannot be used with new.
- Their context (this) is lexically/statically defined, meaning this comes from the surrounding code.

Syntax

```
const functionName = (arg1, arg2, ...) => {
// body of the function
}
```

Basic example

```
const add = (a, b) => a + b;

// Equivalent normal function:
// const add = function(a, b) { return a + b; };

console.log(add(10, 5)); // Output: 15
```

Working: The function takes a and b, calculates a + b, and automatically returns the result because there are no curly braces.

One argument and no argument

```
const double = n => n * 2;
console.log(double(5)); // Output: 10

const hiii = () => alert("Hello");
hiii(); // Shows alert "Hello"
```

Rules: With one parameter, parentheses can be omitted. With no parameters, empty parentheses () are required.

Multiline arrow functions

If the function body has multiple statements, use curly braces. In that case, you must write return explicitly.

```
const add = (a, b) => {
const result = a + b;
return result;
};

console.log(add(5, 5)); // Output: 10
```

Form	Syntax	Returns value?
Expression form	(a, b) => a + b	Yes, automatically
Block form	(a, b) => { ... }	Only with explicit return

Common mistake: Using curly braces without return makes the function return undefined.

11. ES6 Array Methods

JavaScript arrays have built-in methods to add, remove, search, transform, and process data. Important methods include push, pop, shift, unshift, sort, reverse, slice, join, includes, find, some, every, filter, forEach, reduce, and map.

forEach()

- Executes a provided function once for every element in the array.
- Does not return any value.
- Makes code shorter and avoids an extra counter variable.
- Automatically stops when all elements are finished.

```
const numbers = [1, 2, 3];
numbers.forEach(function(num) {
console.log(num);
});
// Output: 1 2 3
```

map()

- Executes a function for every element.
- Returns a new transformed array.
- The new array has the same length as the original.
- Does not change the original array.

```
const numbers = [1, 2, 3];
const doubled = numbers.map(n => n * 2);
console.log(doubled); // [2, 4, 6]
```

reduce()

- Executes a reducer function on each element and produces one single output value.

- The output can be a number, string, object, or array, but it is always one value.
- Callback receives accumulator, current number, index, and array.
- The accumulator holds the running/previous result.
- Helps avoid a global variable for storing previous values.

```
const numbers = [1, 2, 3, 4];
const sum = numbers.reduce((accumulator, number) => accumulator + number, 0);
console.log(sum); // 10
```

Working: Start accumulator = 0. Then: 0+1=1, 1+2=3, 3+3=6, 6+4=10. Final answer is 10.

filter()

- Returns a new array containing all elements that satisfy the test condition.
- Takes a callback function as its first argument.
- Each element is passed to the callback.

```
const numbers = [1, 2, 3, 4, 5];
const even = numbers.filter(n => n % 2 === 0);
console.log(even); // [2, 4]
```

includes()

- Returns true if the array contains the specified value; otherwise false.
- Case sensitive.
- Syntax: array.includes(element, start); element is required, start is optional.

```
const fruits = ["apple", "banana"];
console.log(fruits.includes("banana")); // true
console.log(fruits.includes("Banana")); // false
```

find()

- Returns the value of the first element that passes the test.
- Returns undefined if no element is found.
- Does not execute the function for empty elements.
- Does not change the original array.

```
const numbers = [5, 12, 8, 20];
const firstBig = numbers.find(n => n > 10);
console.log(firstBig); // 12
```

some()

- Checks whether at least one element passes the test.
- Returns true and stops as soon as one element passes.
- Returns false if all elements fail.
- Does not change the original array.

```
const numbers = [1, 3, 8];
console.log(numbers.some(n => n > 5)); // true
```

every()

- Returns true only if the function returns true for all elements.
- Returns false if even one element fails.

- Does not execute the function for empty elements.
- Does not change the original array.

```
const numbers = [2, 4, 6];
console.log(numbers.every(n => n % 2 === 0)); // true
```

Other important methods

Method	Purpose
push()	Adds element(s) at the end of an array.
pop()	Removes the last element.
shift()	Removes the first element.
unshift()	Adds element(s) at the beginning.
sort()	Sorts array elements.
reverse()	Reverses array order.
slice()	Returns a selected part of an array.
join()	Joins array elements into a string.

forEach vs map vs filter vs reduce

Method	Returns	Main use
forEach()	Nothing (undefined)	Perform an action for each element
map()	New array of same length	Transform every element
filter()	New array with matching items	Select elements passing a condition
reduce()	One single value	Combine all values, e.g. sum

12. Destructuring

Simple definition: Destructuring is an ES6 feature that makes it easy to extract values from arrays or properties from objects into separate variables.

Easy analogy: When making a sandwich, you do not take everything out of the refrigerator; you take only the items you need. Similarly, destructuring extracts only the required values from an array or object.

Object destructuring

```
// Before ES6
const person = { name: "Dharmik", age: 25 };
const name = person.name;
const age = person.age;

// After ES6
const { name, age } = person;
```

Rules: The order of names does not matter in object destructuring. The variable name should normally match the object key. You can rename using a colon.

```
const person = { name: "Dharmik", age: 25 };
const { name: userName, age: userAge } = person;
console.log(userName); // Dharmik
console.log(userAge); // 25
```

Array destructuring

```
const colors = ["red", "green", "blue"];
const [first, second, third] = colors;
```

```
console.log(first); // red
console.log(second); // green
```

Skipping items

```
const colors = ["red", "green", "blue"];
const [first, , third] = colors;
console.log(first); // red
console.log(third); // blue
```

Common mistake: In object destructuring, variable names must match keys (unless renamed). In array destructuring, order matters.

13. Spread and Rest Operators

Both use three dots (...), but their purpose is different.

Spread operator (...)

The spread operator allows us to quickly copy all or part of an existing array or object into another array or object. It is often used with destructuring.

```
// Copy array
const arr1 = [1, 2, 3];
const arr2 = [...arr1];

// Clone array so changes do not affect the original
const original = [1, 2, 3];
const clone = [...original];
clone.push(4);
console.log(original); // [1, 2, 3]
console.log(clone); // [1, 2, 3, 4]

// Spread with object
const obj1 = { name: "Amit", age: 20 };
const obj2 = { ...obj1, city: "Rajkot" };
console.log(obj2);
// { name: "Amit", age: 20, city: "Rajkot" }
```

Benefit: A spread-created copy does not refer to the same array, so changing one array does not change the other.

Rest operator (...)

The rest operator collects multiple values into a single array. It is commonly used in array destructuring and function parameters.

```
const [first, ...others] = [1, 2, 3, 4];
console.log(first); // 1
console.log(others); // [2, 3, 4]
```

Spread vs rest

Feature	Spread operator	Rest operator
Symbol	...	...
Direction	Expands values out of an array/object	Collects multiple values into one array
Used in	Array/object copying and combining	Destructuring and function parameters
Example	[...arr1, ...arr2]	const [a, ...rest] = arr

14. Classes in JavaScript

Simple definition: A class is a template or blueprint for creating objects. It groups data (properties) and functions (methods) that work with that data.

- Classes were introduced in ECMAScript 2015 (ES6).
- They provide a cleaner way to follow object-oriented programming patterns.
- Use the keyword class to create a class.
- Always add a constructor() method to initialize object properties.
- A class is not an object; objects are created from it using new.
- JavaScript classes are syntactic sugar over prototypal inheritance; internally they are special functions.

Syntax

```
class ClassName {
constructor() {
// initialize properties
}
}
```

Example

```
class Person {
constructor(name) {
this.name = name;
}

getName() {
return this.name;
}
}

const obj = new Person("Dharmik Aghera");
const ans = obj.getName();
console.log(ans); // Dharmik Aghera
console.log(typeof Person); // function
```

Working: 1) class Person defines the blueprint. 2) new Person("Dharmik Aghera") creates an object and calls constructor(). 3) this.name stores the name in the object. 4) obj.getName() returns the stored name. 5) typeof Person is "function" because JavaScript classes are special functions.

Exam point: A class encapsulates data and functions that manipulate the data.

15. Modules

Simple definition: A module is a separate JavaScript file that organizes related variables and functions so they can be reused in other files.

- Modules help break code into separate files and make the codebase easier to maintain.
- By default, module variables and functions are not available outside the file.
- They must be exported using export and imported using import.
- ES modules work only in strict mode.
- Variables/functions in a module are not globally accessible.

Named exports

A module can have several named exports. While importing, names must match, but they can be renamed using as.

```
// math.js
export const pi = 3.14;
export function add(a, b) {
return a + b;
}

// main.js
import { pi, add } from "./math.js";
import { add as sum } from "./math.js";
```

All named exports can be imported into one object using *.

```
import * as math from "./math.js";
console.log(math.add(2, 3));
```

Default exports

A module that needs to export only one main value can use a default export. There can be only one default export per module. A default export can be imported with any name.

```
// greet.js
export default function greet() {
return "Hello";
}

// main.js
import anyName from "./greet.js";
console.log(anyName()); // Hello
```

Named vs default export/import

Feature	Named	Default
Number per module	Many	Only one
Export syntax	export const x = 1;	export default value;
Import syntax	import { x } from "./file.js";	import anyName from "./file.js";
Name while importing	Must match, unless renamed with as	Can be any name

16. Final Revision

Chapter-wise revision summary

Topic	Remember this
DOM	Interface to access/update HTML content, structure and style; window is the global object.
document methods	getElementById() returns one element; getElementsByName() and getElementsByTagName() return collecti
Forms	Access values as document.forms["formName"].fieldName.value.
DHTML	HTML + CSS + JavaScript for dynamic pages and effects.
Events	onclick, ondblclick, onmouseover, onkeydown, onkeyup, onload, onfocus, onblur, onchange, onsubmit.
let	Block scope; must declare before use; cannot redeclare in same scope.
const	Block scope; cannot reassign; must initialize; array/object contents can change.
Arrow function	Short function syntax using =>; anonymous; no prototype; cannot be used with new.
forEach	Loops through elements; returns nothing.
map	Transforms every element; returns a new array of the same length.
filter	Returns a new array of elements passing a condition.
reduce	Combines array into one single value using an accumulator.
includes	Returns true/false if a value exists; case sensitive.
find	Returns first matching value or undefined.
some/every	some = at least one passes; every = all must pass.
Destructuring	Extracts array values/object properties into variables.
Spread	Expands array/object values; used for copying and merging.
Rest	Collects remaining values into an array.
Class	Blueprint for objects; uses class and constructor(); internally a special function.
Modules	Separate files using export and import; named and default exports.

Last-minute checklist

- Can you explain DOM and the role of window?
- Do you know the difference between getElementById, getElementsByName and getElementsByTagName?
- Can you list mouse, keyboard, frame/object and form events?
- Can you write let, const and var examples and state their scope rules?
- Can you convert a normal function into an arrow function?
- Do you know which array methods return a new array and which return one value?
- Can you perform object and array destructuring?
- Can you distinguish spread from rest?
- Can you create a class with constructor and methods?
- Can you write named and default export/import examples?

17. Exam Practice Questions and Answer Key

Short-answer questions

Q. What is the DOM?

Ans. The Document Object Model is a platform- and language-neutral interface that allows programs and scripts to dynamically access and update the content, structure, and style of documents.

Q. What is the window object?

Ans. The window object represents the browser window/frame displaying the document. It is the global object in client-side JavaScript, and all client-side objects are connected to it.

Q. What does getElementById() return?

Ans. It returns the single HTML element whose id matches the specified value.

Q. What is DHTML?

Ans. DHTML (Dynamic HTML) is a combination of HTML, CSS, and JavaScript used to create dynamic and interactive web pages.

Q. What is an arrow function?

Ans. An arrow function is a shorter, anonymous function syntax in ES6 that uses the => symbol.

Q. What is destructuring?

Ans. Destructuring is an ES6 feature used to extract values from arrays or properties from objects into separate variables.

Q. What is the spread operator?

Ans. The spread operator (...) expands/copies all or part of an existing array or object into another array or object.

Q. What is a JavaScript class?

Ans. A class is a template/blueprint for creating JavaScript objects. It encapsulates data and functions that manipulate that data.

Q. What is a module?

Ans. A module is a chunk of JavaScript code in a file that organizes related variables and functions; its components must be exported to be used in other files.

Q. Difference between some() and every()?

Ans. some() returns true if at least one element passes the test; every() returns true only if all elements pass the test.

Long-answer questions

Q. Explain DOM with its properties and methods.

Ans. The DOM is a tree-like representation of an HTML document that JavaScript can access and modify. The window object is the global entry point; document is accessed through it. Important document properties include body, cookie, domain, forms, images, title, and URL. Important methods include write(), writeln(), open(), close(), getElementById(), getElementsByName(), getElementsByTagName(), setTimeout(), and clearTimeout(). JavaScript can use these to read page information and dynamically change elements.

Q. Explain event handling in JavaScript with categories and examples.

Ans. Events are user or browser actions such as clicks, key presses, and form submissions. JavaScript responds using event attributes. Mouse events include onclick, ondblclick, onmousedown, onmousemove, onmouseover, onmouseup, and onmouseout. Keyboard events include onkeydown, onkeypress, and onkeyup. Form events include onfocus, onblur, onchange, onreset, onselect, and onsubmit. Frame/object events include onload, onresize, onscroll, onerror, and onabort. Example: Click displays an alert when clicked.

Q. Explain var, let and const with examples.

Ans. A variable stores a value in memory. var is function-scoped and can be redeclared and reassigned. let is block-scoped, must be declared before use, and cannot be redeclared in the same scope. const is block-scoped, must be initialized, and cannot be reassigned; however, properties of a const object or elements of a const array can still be changed. Example: let x = 5; x = 10 is valid, while const y = 5; y = 10 gives an error.

Q. Explain arrow functions with syntax and examples.

Ans. Arrow functions provide shorter function syntax using =>. A one-line arrow function such as const add = (a,b) => a+b automatically returns a+b. With one parameter, parentheses may be omitted: const double = n => n*2. With no parameters, empty parentheses are required: const hi = () => alert("Hello"). For multiple statements, curly braces are used and an explicit return is needed. Arrow functions are anonymous, cannot be constructors, have no prototype property, and have lexically defined this.

Q. Explain forEach, map, filter and reduce with examples.

Ans. forEach executes a function for each array element but returns nothing. map executes a function for each element and returns a new transformed array of the same length. filter returns a new array containing only elements that satisfy a condition. reduce applies a reducer function to each element and returns one single value, using an accumulator to remember the previous result. Example: [1,2,3].map(n=>n*2) gives [2,4,6]; [1,2,3,4].filter(n=>n%2===0) gives [2,4]; [1,2,3,4].reduce((a,n)=>a+n,0) gives 10.

Q. Explain classes in JavaScript with an example.

Ans. A JavaScript class is a blueprint for creating objects, introduced in ES6. It is created with the class keyword and usually contains a constructor() to initialize properties. Methods are defined inside the class. Example: class Person { constructor(name) { this.name = name; } getName() { return this.name; } }. Creating new Person("Dharmik") calls the constructor and stores the name. Calling obj.getName() returns it. JavaScript classes are syntactic sugar over prototypal inheritance and are internally special functions.

Q. Explain ES6 modules, export and import.

Ans. A module is a JavaScript file containing related variables and functions. By default, its members are private to that file. Named exports allow multiple exports and must be imported using matching names, though they can be renamed with as. Default exports allow one main export per module and can be imported with any name. Modules make code reusable, organized, and easier to maintain. Example: export function add(a,b){return a+b;} in math.js, then import { add } from "./math.js"; in another file.

MCQs

1. Which object is the global object in client-side JavaScript?

A. document B. window C. body D. form

Correct answer: B. window is the global object and the entry point to browser features and the document.

2. Which method returns all elements with a specified tag name?

A. getElementById() B. getElementsByName() C. getElementsByTagName() D. querySelector()

Correct answer: C. It returns a collection of elements with the given tag name.

3. Which keyword creates a block-scoped variable that can be reassigned?

A. var B. let C. const D. static

Correct answer: B. let is block-scoped and allows reassignment.

4. What is the output of const add=(a,b)=>a+b; console.log(add(10,5));?

A. 105 B. 15 C. undefined D. Error

Correct answer: B. The arrow function adds 10 and 5.

5. Which array method returns a new transformed array?

A. forEach() B. map() C. reduce() D. find()

Correct answer: B. map returns a new array of the same length.

6. Which method returns true only if all elements pass a test?

A. some() B. find() C. every() D. filter()

Correct answer: C. every() requires all elements to pass.

7. Which method returns the first element that passes a test?

A. filter() B. find() C. some() D. includes()

Correct answer: B. find() returns the first matching value.

8. Which symbol is used for spread and rest operators?

A. .. B. ... C. ** D. &&

Correct answer: B. Both use three dots.

9. How many default exports can a module have?

A. Zero B. One C. Many D. Unlimited

Correct answer: B. A module can have only one default export.

10. What does typeof Person return for a JavaScript class?

A. object B. class C. function D. undefined

Correct answer: C. JavaScript classes are special functions.